CARVE—A Constructive Algorithm for Real-Valued Examples

Steven Young and Tom Downs, *Member, IEEE*

***Abstract*— A constructive neural-network algorithm is presented. For any consistent classification task on real-valued training vectors, the algorithm constructs a feedforward network with a single hidden layer of threshold units which implements the task. The algorithm, which we call CARVE, extends the "sequential learning" algorithm of Marchand *et al.* from Boolean inputs to the real-valued input case, and uses convex hull methods for the determination of the network weights. The algorithm is an efficient training scheme for producing near-minimal network solutions for arbitrary classification tasks. The algorithm is applied to a number of benchmark problems including Gorman and Sejnowski's sonar data, the Monks problems and Fisher's iris data. A significant application of the constructive algorithm is in providing an initial network topology and initial weights for other neural-network training schemes and this is demonstrated by application to backpropagation.**

***Index Terms*—Constructive algorithms, convex hulls, internal representations, linear separability, Monks problem, polynomial time complexity, Sonar data.**

I. Introduction

THERE is currently no satisfactory general answer to the question of how to choose a network architecture for a given application. As a consequence, considerable effort has been applied to the development of techniques for incrementally modifying a network so that it accommodates a given task. These techniques fall into two categories: 1) pruning techniques and 2) constructive techniques. For pruning techniques, a large network is trained and then is pruned back by removing network connections and/or hidden units. Not only does pruning lead to a reduction in network complexity, but one expects additional benefits in the form of reduction of any overfitting that occurs in the large network and an improvement in generalization performance. Some of the best known pruning techniques are described in [1]–[3]. In contrast to pruning methods, constructive techniques commence with a small network and, during training, increase the size of the network when necessary in order to accommodate the learning task. A large variety of constructive techniques have appeared in the past few years. For recent reviews see [4] and [5].

This paper describes a new constructive technique that builds a neural network with a single hidden layer of threshold units. This technique is applicable to classification tasks where training examples are classified into any number of classes and is able to implement correctly any consistent classification task on a finite training set. A number of constructive schemes have already appeared in the neural-networks literature which have similar guarantees on network construction but most of these (see, for instance [6]–[11]) are only applicable to problems with Boolean-valued inputs. The technique we describe here is not restricted in this way, being capable of handling both real-valued and Boolean-valued inputs. We call our method "CARVE," which stands for "constructive algorithm for real-valued examples." CARVE is an extension of the sequential learning algorithm described in [6] to the case of real-valued inputs. This extension to real-valued inputs is achieved by the use of a convex hull search method and this constitutes the essential difference between CARVE and the sequential learning algorithm. This search allows hyperplanes to be found which separate substantial sets of points (all of one class) from the remaining training examples. These "carve sets" are separated from the remaining examples by a suitable hidden unit.

A. Related Techniques

Within the pattern recognition literature there are a number of pattern classification methods to which the CARVE method has some similarities. The architecture for CARVE is a two layer network of threshold units, where the size of the hidden layer is increased until there are sufficient units for the training task. This architecture is used in other pattern classification techniques, e.g., the MADALINE [12] which was introduced as a fixed rather than constructive architecture.

CART-based methods [13] split data according to various types of rules and conditions of purity on the branches of the decision trees that they construct. A CART method which required "pure" nodes would be quite similar to CARVE, though the method for finding carve sets that we present is novel.

This paper is organized as follows. Section II presents the basic CARVE algorithm showing the convergence of the algorithm. Section III describes the development of methods that find carve sets—this is the central work of the algorithm; the methods we consider are compared with previous techniques. We address the question of how extensive the search around convex hulls needs to be and present the results of analysis which shows that our algorithm has polynomial time complexity. We also describe the extension of CARVE to the case where the hidden units have higher-order threshold characteristics, as well as the use of networks found by CARVE

Manuscript received April 20, 1996; revised August 7, 1997.

S. Young is with the Department of Experimental Psychology, University of Oxford, OX1 3UD, U.K.

T. Downs is with the Department of Electrical and Computer Engineering, University of Queensland, Brisbane, Australia 4072.

Publisher Item Identifier S 1045-9227(98)08517-8.

1. Begin with a network which has an empty hidden layer, and one output unit for each class in the training set.
2. Find a set of points of the same class that can be separated by a hyperplane from the other training examples. Call this set of points the *carve set.*
3. Install a hidden unit that implements a hyperplane which separates the carve set found in step 2. Remove the carve set from the training set.
4. Repeat steps 2–3, each time reducing the training set, until only points of one class remain in the training set. When this occurs, the hidden layer is complete.
5. Determine the hidden to output layer weights.

Fig. 1. The CARVE algorithm.

as initial networks for backpropagation training. Section IV presents results from the application of CARVE to a broad range of classification tasks.

II. The CARVE Algorithm

The CARVE algorithm begins with an empty hidden layer into which threshold units are added one at a time until the layer is complete. A threshold unit implements a hyperplane in the input domain and the aim is to find a hyperplane that separates a set of points of one class only from the remaining training examples. The set of points that is separated by the hyperplane (the *carve set*) is removed from the training set. The next unit to be added to the network aims to separate another set of points of one class only, but now only from the reduced training set. Once a carve set for this unit is found, the unit is added to the layer and the carve set is removed from the training set. The construction of the hidden layer continues with each subsequent unit implementing a hyperplane that 'carves' points of one class from the remaining training examples. The hidden layer is complete when only points of one class remain in the training set. Once the hidden layer is complete, the output unit weights must be determined, to complete the network construction. There will be one output unit for each class in the training set, each of which is ON for training examples of the corresponding class and OFF otherwise. Note that the output units receive input from the hidden layer only (ie. there are no direct connections from the input units). A formal description of the algorithm is given in Fig. 1. An example network construction is given in Fig. 2. In Fig. 2(b), the first hidden unit separates a carve set of black points. These are removed from consideration (in Fig. 2(c) they are turned grey), and the second hidden unit separates a set of the white points. The third hidden unit separates the cross points from the remaining black points in the training set. Output units are introduced for each class and this completes the network construction.

The algorithm description given above is not complete in that the methods for some steps are not completely specified and in fact it is not immediately clear that certain steps are always possible. The questions that need to be addressed are: 1) Is it always possible to find a carve set? 2) As the training set is reduced will the situation eventually arise where only points of one class remain in the training set? 3) Do appropriate hidden to output unit weights exist for the hidden layer constructed by the CARVE algorithm? This last question can be addressed by answering question 4): Are the hidden unit

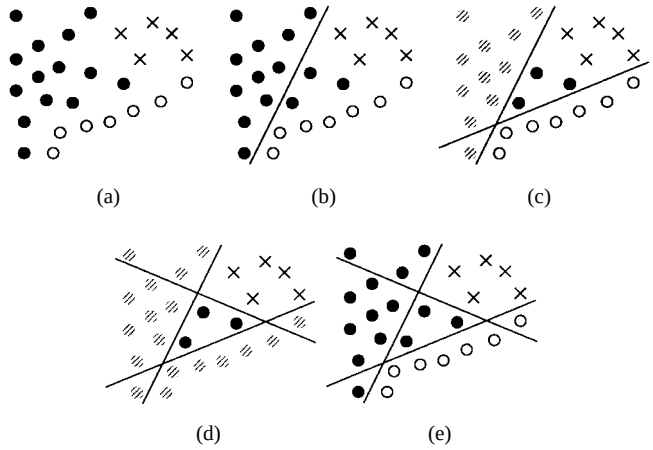

Fig. 2. CARVE construction of a network.

representations of the network *faithful* and *linearly separable*? Questions 1) and 2) are straightforward. To show that it is always possible to find a carve set, we require the restriction that the training task is consistent, that is, any single training example is not classified into more than one class. For 1), with a consistent training set, it can be seen that there is always at least one extremal point that can be separated from the other training examples by a hyperplane. For 2), it is easy to see that the reduced training set will eventually have points of one class, because, given that a carve set can always be found, the finite training set will ultimately reduce to a set of only one class of points.

For question 4), and hence 3), to show that the hidden unit representations are faithful, we need only note that the CARVE construction separates training vectors of different classes, so that the partitioning of input space by the hyperplanes of the hidden units must be faithful and hence the hidden unit representations must also be faithful. It remains to show that the hidden unit representations are linearly separable, which we address in the next section.

A. The Hidden Unit Representations Are Linearly Separable

Having hidden unit representations that are faithful is a necessary but not sufficient condition for being able to construct a single hidden layer network which correctly classifies a training task. To ensure that the output units can correctly classify the hidden unit representations we also require that the representations for each class in the training set be linearly separable from the representations of other classes.

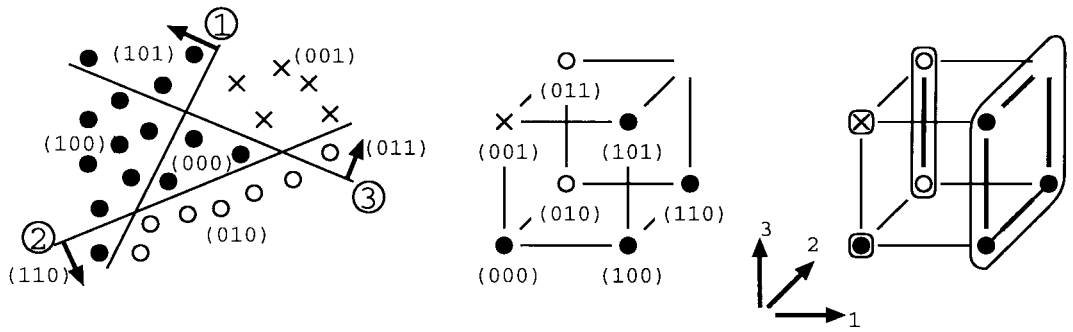

Fig. 3. Hidden unit representations of CARVE construction example.

Marchand *et al.* [6] showed that the hidden unit representations generated by a CARVE-like procedure are linearly separable by giving an explicit formula for determining the weights. The weight values generated by this formula grow exponentially with the number of hidden units. An improved formula for the hidden layer to output unit weights is given by Muselli in [11], where the weight values grow less quickly.

We here present an alternative method for showing that the hidden unit representations are linearly separable. We do this by considering the way in which the CARVE construction splits up the hidden unit representations into sets of Boolean vectors.

Consider the hidden unit representations of the example given in Fig. 2. There are three hidden units so the hidden unit representations are three-dimensional (3-D) Boolean vectors. In Fig. 3(a) we show the CARVE construction from Fig. 2 with hidden unit numbers and arbitrary directions given to the hyperplanes to specify the hidden unit representations. In Fig. 3(b) the hidden unit representations are shown as the vertices of a three-cube. For this example, it is easy to see that the representations for each class, $\bullet, \circ$, and $\times$, are linearly separable from the representations of the other classes, since hyperplanes can be found that separate the hidden unit representations of each class from the representations of the other classes.

Now consider the hidden unit representations of each carve set. Any training example in the carve set of the first hidden unit has hidden unit representations of $(1 * \ *)$ where $*$ stands for either zero or one (Here the representation (111) is not present since the input space partition has no cell that corresponds to this representation). The second carve set corresponds to the hidden unit representations $(01*)$, and the third carve set has hidden unit representation (001), with the remaining points in the training set having hidden unit representation (000). Notice that the hidden unit representations corresponding to each carve set split the three-cube in a regular fashion. Specifically, the first carve set corresponds to representations on the right face of the cube, the second carve set corresponds to the back left edge of the cube and the third carve set to the top front left vertex, leaving the bottom front left vertex corresponding to the remainder of the training set at the end of the CARVE construction. For the more general situation where a CARVE construction has introduced m hidden units into a hidden layer, we can describe the partition of the hypercube of hidden unit representations analogously: The outputs of the layer of m hidden units correspond to m-dim Boolean vectors which can be visualized as the vertices of the m-cube. The first hidden unit separates the m-cube into two $(m-1)$-cubes. One of the $(m-1)$-cubes contains hidden unit representations that correspond to the first carve set. The second hidden unit then splits the other $(m-1)$-cube into two $(m-2)$-cubes and so on for subsequent units. [The splitting of the m-cube for $m=3$ is shown in Fig. 3(c)]. The hidden unit representations which correspond to one class can therefore be described as a collection of subcubes, obtained by partitioning the hypercube as described above. To show that the hidden unit representations for any one class are linearly separable from the hidden unit representations of the other classes, we show that all possible collections of these subcubes form sets of Boolean vectors that are linearly separable from their complement in the hypercube. First note that the orientation of the partitioning of the hypercube is unimportant, since any relabeling of the axes will not affect the separability of the set of vectors. Call a set of m-dim Boolean vectors a *linearly separable set* if the set and its complement in the m-cube is able to be separated by an $(m-1)$-dim hyperplane. With these definitions we prove the following theorem.

Theorem 1: All possible collections of subcubes of a CARVE partition of the m-dimension hypercube are linearly separable sets.

Proof: The proof is by induction on m**.** For $m=1$, we need only consider the separation of two zero-cubes and it is clear that a straight line can always be found to implement this task. Hence the theorem holds for $m=1$. For the inductive step, consider the $(m+1)$-cube. The first hyperplane introduced by CARVE splits the $(m+1)$-cube into two m-cubes. Further applications of CARVE partition one of these m-cubes into smaller cubes, but the other m-cube remains unaffected. Refer to the unchanging m-cube as the *primary m-cube*, and the one that is further partitioned by CARVE as the *secondary m-cube*. Then, as CARVE proceeds, it partitions the secondary m-cube into an $(m-1)$-cube, an $(m-2)$-cube, $\cdots$, a two-cube, a one-cube, and two zero-cubes. Now, by assumption, the subcubes created by CARVE's partitioning of the secondary m-cube are linearly separable. But also, in the vectors represented by the vertices of the primary m-cube there is one particular vector entry (the same one in each case) that we know has a different value from the same

vector entry in the vertices making up the secondary m-cube. This implies that we can find a linear threshold function (by suitable choice of weight values) to separate from the primary m-cube any collection of subcubes in the secondary m-cube. Conversely, note that if a set of Boolean vectors is linearly separable then its complement is also, so since the collections of subcubes from the $(m+1)$-cube which do not contain the primary m-cube are linearly separable, then the collections of subcubes which do contain the primary m-cube are also linearly separable. This shows that all collections of subcubes in the $(m+1)$-cube are linearly separable if the collections of subcubes of the m-cube are linearly separable, hence completing the proof. ■

The hidden unit representations, then, satisfy faithfulness and linear separability conditions. These are sufficient conditions for the implementation of a classification training task by a single hidden layer network. In practice, we use linear programming to find the hidden to output weight values since we are assured that the hidden unit representations are linearly separable. The CARVE algorithm will successfully construct an appropriate single hidden layer threshold network for any consistent classification task. In this section, we have shown that carve sets can always be found, but the actual methods we employ for finding carve sets remain to be discussed. We consider these methods in the following section.

III. Finding CARVE Sets

A. Basic CARVE

The task of finding carve sets is the main work of the CARVE algorithm. Suppose the training set contains k classes of points; represent the set of points in each class by $S_i, i = 1, 2, \cdots k$. The CARVE algorithm seeks to identify the largest carve set for each class.

Let H_i represent the set of all points in the training set except those in S_i; so $H_i = \bigcup_{j \neq i} S_j$. For each i, the CARVE algorithm seeks the largest set $C_i \subset S_i$ for which a hyperplane boundary separates the set C_i from all points in H_i. We refer to H_i as the hull set for S_i.

Unfortunately, as noted in [14], the problem of finding the largest carve set is NP-hard because it contains the densest-hemisphere problem [15] as a special case. This makes any optimal search method too expensive computationally; however, as we show below, the task of finding individual carve sets is far more tractable and this allows us to develop a polynomial-time algorithm that determines good (but not necessarily optimal) carve sets.

In [6] and [14], Marchand *et al.* described two techniques for finding carve sets. Both involved establishing the linear separability of subsets of points from other points in the training set. The method used in [6] was based upon the pocket algorithm and its performance was hampered by the fact that the pocket algorithm does not converge in the case where points are not linearly separable. The second paper [14] described an improved method that employed linear programming to determine whether specific subsets of the training set are linearly separable. A drawback of this method is that the number of possible subsets of points that could be considered for linear separability is exponential in the size of the set (a set of n points has 2^n such subsets). In an attempt to circumvent this problem, Marchand *et al.* employed some problem-specific heuristics, but for their methods, the size of the search space still appears to be a problem. In [10], Frattale-Mascioli and Martinelli provide a novel method for finding "binary cuts" of sets of Boolean vectors as well as a scheme for converting from real-valued vectors to Boolean vectors, but our objective is to work directly with real-valued vectors.

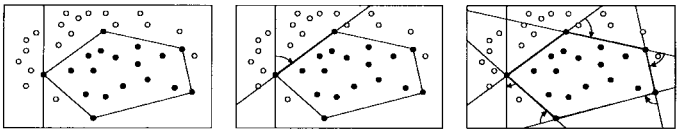

Fig. 4. Traversing the boundary of a 2-D convex hull.

Our own approach to the search for carve sets avoids the issue of testing for linear separability of subsets of points by directly searching for hyperplanes that separate sets of points of one class only. Note that only points outside a convex hull can actually be separated by a hyperplane boundary from the hull set. So to find the largest carve sets, we need only consider hyperplanes that touch the boundary of the convex hull. The hyperplane with the largest set of points in its open halfspace outside the convex hull is the largest carve set. In this way, the largest carve set and a suitable hyperplane can be found and the weights for the hidden unit to implement the hyperplane can be directly determined.

In the two-dimensional (2-D) case, the convex hull is a convex polygon. An example is shown in Fig. 4. To find hyperplanes that separate the points within the hull from points outside the hull, we can use the following method: Consider initially a line which passes through a boundary point or vertex of the polygon and which does not pass through the polygon. Such a line is easy to find; for example, choose a line parallel to the vertical axis of the plane, which passes through the point at the left extreme of the hull set [as shown in Fig. 4(a)]. Rotate this initial line around the boundary point until it passes through another point of the hull set [Fig. 4(b)]. This point will also be a boundary point and the two boundary points that the line now intersects form a boundary edge of the polygon. Rotate the line around the new end point, until the next boundary point is encountered. These rotations can be continued until the line has returned to its initial position. As the line traverses the hull, it passes through all the hyperplane boundaries that touch the boundary of the hull [Fig. 4(c)]. This method of searching the boundaries of the 2-D convex hull is analogous to the 2-D version of the *gift wrapping* method for constructing a convex hull [16]. As the line is rotated, the carve sets that the line separates can be established and the position of the line that yields the largest carve set can be determined.

Note that since polynomial time algorithms exist for determining the convex hull of a set of 2-D points, then, in the 2-D case, it is always possible to find the largest CARVE set in polynomial time.

In the case where the input dimension $d \geq 3$, the situation is more complex. Traversal of the convex hull is not as simple as in the 2-D case but can be dealt with using the general dimension gift wrapping method [17], [18]. This method for finding convex hulls can be described as a two-stage process as below:

STAGE 1

Step 1: Find a hyperplane that passes through a boundary point (or boundary points) of the hull. This is done by choosing a random direction and determining the hyperplane normal to this direction that passes through a boundary point (or points) of the hull and has the remainder of the hull set in one halfspace. We call this the *initial hyperplane*, and the boundary point (or points) the hyperplane passes through is called the *boundary set*.

Step 2: Choose another random direction and determine another hyperplane, normal to that direction, which passes through a point (or points) of the boundary set and which has the remainder of the hull set in one halfspace. Rotate the initial hyperplane around its intersection with the second hyperplane until the hyperplane touches another point (or points) in the hull set. This point (or points) is (are) added to the boundary set.

Step 3: Check whether the boundary set constitutes a $(d-1)$-dimensional boundary face.[1] If it does, go on to Stage 2; if it does not, go back to Step 2.

STAGE 2

The full set of $(d-1)$-dimensional boundary faces that touch the boundary face determined in Stage 1 can now be found. The full details need not concern us here, but the procedure involves rotating hyperplanes around the $(d-2)$-dimensional subfaces of the boundary face determined in Stage 1. Extending this procedure further, the complete convex hull can be determined; for details, see [18].

In the implementation of the CARVE algorithm, we do not concern ourselves with finding all the boundaries of the convex hull (their number may be exponential in the dimension of the pattern space), but rather with finding large carve sets. So our procedure involves only part of the gift-wrapping scheme, namely Step 1 and a slightly modified version of Step 2 in Stage 1 of the procedure described above.

The modifications that we make to Step 2 are as follows.

1) We rotate the initial hyperplane about its intersection with the second hyperplane, *in both of the available directions*, until it makes contact with the hull set.
2) We determine the sets of points encountered by the hyperplane during its rotation—these are potential carve sets.
3) We do not add points to the boundary set of the initial hyperplane—this is unnecessary.

We will refer to the modified version of Step 2 as Step 2′.

In order to build up a profile of potential carve sets, Steps 1 and 2′ are repeated a prescribed number of times. The number of times we implement Step 1, which determines an initial hyperplane, is denoted by NInit and the number of times we implement Step 2′ for each initial hyperplane is denoted by NRot. Thus there is a total of NInit $\times$ NRot hyperplane rotations involved in the process of determining a carve set. And after these rotations have been carried out, we select as our carve set that set C_i which maximizes $|C_i|/|S_i|$, where C_i and S_i are as defined at the beginning of this section. In common with the sequential algorithm in [6], our selection criterion chooses the carve set that constitutes the largest proportion of a class of points that can be separated.[2]

It is straightforward to show [19] that the worst-case time complexity of the CARVE algorithm is $O(n^2(\log n + d)\times$ NInit $\times$ NRot), where n is the number of training examples and d, as above, is the dimension of the set of training examples. The values of NInit and NRot are selected by the user of the algorithm. Clearly for larger values of the product NInit $\times$ NRot, the more likely one is to find large carve sets and hence the more likely it is that one's network realization will have a relatively small number of hidden units. The experimental results in Section IV bear this out but also indicate a law of diminishing returns, such that increases in the value of the product NInit $\times$ NRot eventually cease to have any significant effect.

B. Higher Order CARVE

The basic CARVE algorithm described in the previous section employs linear threshold units to provide the hyperplanes which separate sets of points of the same class from the remaining points. In this section, we briefly discuss how the basic algorithm can be extended to higher order threshold units.

Extension to the higher order case is straightforward. Note that higher-order threshold functions consist of weighted sums of polynomial terms in the input variables, i.e., weighted sums of $x_1, x_2, x_3, \cdots, x_1^2, x_1x_2, x_1x_3, \cdots, x_2^2, \cdots$, etc. The set of polynomial terms can be considered to be a transformation of a vector into a higher dimensional space. The training data can be transformed into this higher dimensional space and convex hulls of the transformed data points. It follows also that convex hull boundary search methods can also be used.

Although the extension to higher order CARVE is straightforward, it does have practical limitations because of the rapidity with which the number of polynomial terms increases as we increase the order of the solution we seek.[3] One possible way to avoid the explosion in the dimensionality of the transformed data points is to use only a (suitably selected) subset of the polynomial terms. An illustration of the effect of such action is given in terms of the two-spirals problem in the next section.

[1] This can be done by determining whether the rank of the matrix formed by the points in the boundary set is at least $(d-1)$.

[2] Choosing the largest proportion rather than the largest set can have an advantage. Consider the case where the number of points in a class r is small and a carve set C_r constituting the whole class is found. It may be that a larger carve set C_s (with a lower proportion $|C_s|/|S_s|$) can be found, but it is preferable to choose C_r because this removes class r from the training set and brings it closer to the hidden layer completion criterion of having only points of one class remaining in the training set.

[3] The number of terms in a polynomial threshold function of nth order with d inputs is $\binom{n+d}{d}$

C. CARVE-BP

Since CARVE always generates correct solutions to any consistent classification task, it provides a way of determining the size of a network which is sufficient for solving classification tasks. By applying CARVE with increasing values of (NInit, NRot) the minimal network size that CARVE produces can be found. The resulting network structure can then be trained using backpropagation (BP) or some other training scheme, knowing that a solution exists for this network size. Also the network weights that CARVE finds can be used as initial network weight vectors for BP training. CARVE uses threshold units, whereas BP uses continuous activation functions, normally sigmoidal units. Note that for a threshold network the magnitude of the input weight vector to each unit of the threshold network is unimportant—only the direction of the weight vector, which is normal to the hyperplane that the unit implements in its input space, determines the output of the threshold unit. This allows us to adjust the magnitude of the network weights without affecting the output of the threshold network. When the weights for threshold networks are used as initial weights for a sigmoidal network, it is not necessarily the case that the sigmoidal network will give the same output values as the threshold network. If the magnitude of the input weights to each unit of the network is large then the sigmoidal units approximate threshold functions and the sigmoidal network will produce equivalent output to the threshold network. Applying BP to such a network will not effect much change because the training set error will already be minimal. However, if the weight magnitude is decreased, this will cause the sigmoidal network to generate outputs that differ from those of the threshold network and application of BP will then generate different solutions which utilize the continuous nature of the sigmoidal activation functions. So a factor in utilising CARVE networks as initial weights for a BP network is the magnitude of the weight vector that we pass on to the BP algorithm. In practice the CARVE algorithm returns networks with weight vectors normalized to length one. We introduce a *weight factor* parameter, where the weight values are multiplied by the weight factor. In general, we use a weight factor of one for our experiments unless otherwise stated.

IV. Experimental Results

For any classification task over a finite set of training examples CARVE will generate a single hidden layer threshold network which correctly classifies the training set. In this section we present experimental results from the application of CARVE to a number of classification problems. In this presentation we look specifically at two important aspects of our algorithm's performance, namely 1) the size of the networks it generates and 2) the generalization performance achieved.

Note that because the CARVE procedure depends upon randomly-generated hyperplanes, multiple applications of the procedure to the same learning task will normally lead to the production of different networks and, in particular, to the production of networks of different sizes. And so, in the experiments to be described below, we have run multiple trials on each learning task reporting the average size of network generated. In addition, we have experimented with the values of the parameters NInit and NRot in order to assess their effect upon network size.

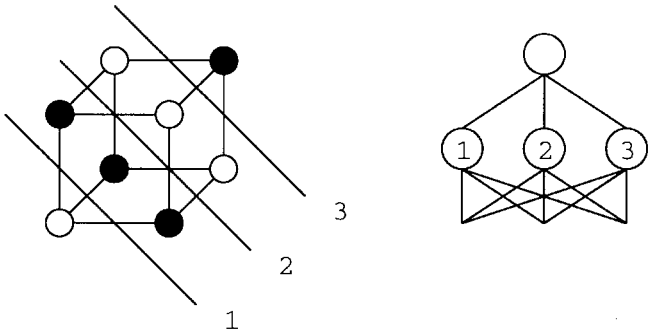

Fig. 5. Three-bit parity and three hidden unit network solution.

The first two learning tasks below concern the learning of Boolean functions and we use these simply as vehicles for comparing the size of network produced by CARVE with network size obtained using other well-known techniques. The other four learning tasks that we consider are employed to compare the performance of CARVE with other techniques both in terms of the size of networks produced and their generalization ability.

A. Parity

The parity problem has been used to test many neural-network classification algorithms. The problem is to classify Boolean input vectors according to whether or not the input vector has an odd number of elements with value one (i.e., odd parity).

The training set for the n-bit problem consists of all 2^n Boolean inputs and it is well-known that a single hidden layer solution exists with n hidden units [20]. Fig. 5 illustrates the three hidden unit solution for the three-bit parity problem. The solution has hidden units that implement hyperplane boundaries that are generally parallel to each other and separate sections of the hypercube with input vectors of the same class.

A number of other constructive algorithms which build layered networks have been applied to parity [6], [7], [21] and they have usually been successful in finding an n-hidden unit solution. (In [7], the Tiling algorithm is reported to have produced the n unit solution for $n = 1, \cdots, 9$, but produced a network with 11 hidden units for $n = 10$).

The experiments we carried out with CARVE indicate that it is capable of finding the n hidden unit solution so long as the search parameters (NInit, NRot) are given sufficiently large values. The values of NInit and NRot required for the algorithm consistently to produce the n unit solution, for $n = 4, \cdots, 10$, are shown in Table I.

B. Random Boolean Functions

A random n-bit Boolean function is a training problem consisting of all 2^n Boolean vectors with each training vector being randomly assigned to one of two classes with equal probability. We applied CARVE to the random Boolean func-

TABLE I
VALUES OF NInit, NRot FOR CARVE TO CONSISTENTLY PRODUCE n UNIT SOLUTIONS FOR n-BIT PARITY

n	(NInit, NRot)
4	(15,15)
5	(25,25)
6	(45,45)
7	(55,55)
8	(75,75)
9	(140,140)
10	(190,190)

tion task, and in Table II the average network sizes generated by CARVE over 100 trials are given, along with standard deviations.

The network sizes were obtained after experimenting with values of (NInit, NRot). For $n = 4, 5$, and 6, the network sizes given in Table II were obtained with values of (NInit, NRot) less than (50, 50), but the network sizes given for $n = 7, 8$ were obtained with (NInit, NRot) = (70, 100). Table II also contains results reported for four other constructive algorithms that have been applied to this task. The value in the angle braces at the top of each column is the number of trials over which the network size is averaged. CARVE produces smaller networks for this classification task than any of these other constructive algorithms listed here. Significantly CARVE produces smaller networks than the sequential learning algorithm of Marchand *et al.* This indicates that CARVE is able to find larger carve sets than the sequential learning method.

C. Two Spirals

The two spirals problem [22] is a classification task which is highly nonlinearly separable. Single hidden layer networks trained by backpropagation generally fail to produce solutions to this problem and constructive algorithms have been more successful [23]. The CARVE algorithm applied to the two spirals task generates single hidden layer threshold network solutions. Example solutions are given in Fig. 6.

In contrast to our experience with the two previous learning tasks, the average network size generated by CARVE on this problem does not decrease as (NInit, NRot) increases but instead stays within the same range of values. A typical example is (NInit, NRot) = (10, 10), for which the average network size obtained (over 100 trials) was 34.34 ± 4.14, with a minimum network size of 25 hidden units and maximum size of 43 hidden units. This range of network sizes obtained for the two spirals problem improves upon the 50 unit single hidden layer network solution reported by Baum and Lang for their query learning scheme [24].

As can be seen in Fig. 6, the decision regions of the two solutions do not approximate the spiral regions very accurately. Because of the highly nonlinear nature of this problem, it is likely that better solutions could be obtained by using CARVE with higher order threshold units. For higher order CARVE we found that the network size of solutions to two spirals again decreases as (NInit, NRot) parameter values increase. The smallest second-order solutions had seven hidden units and were obtained consistently for (NInit, NRot) values above (35, 35). The average size for third-order solutions with (NInit, NRot) = (100, 100) was 10.0, with minimal network size having eight hidden units. (Our results with second-order units imply that third-order solutions with seven and possibly fewer hidden units should exist, but these solutions were not found by CARVE for the values of (NInit, NRot) that we tried.) The minimal size obtained using fourth-order solutions was six hidden units, but networks of this size were not obtained consistently. For values of (NInit, NRot) larger than (50, 50) the average size of fourth-order network solutions was seven hidden units. Examples of these higher order solutions are given in Fig. 7. The smoothness of higher order polynomial threshold surfaces seems well suited to the two spirals task.

Holt [25], [26] has produced a constructive algorithm that appears particularly suited to the two spirals problem in that it employs hidden units with hyperspherical decision boundaries. Holt's method applied to the two spirals problem produced networks with an average of 9.5 hidden units (minimum: eight, maximum: 13). Higher order CARVE (with the hyperspherical restriction; i.e., second order, but ignoring the term x_1x_2 and forcing the weights for the x_1^2 and x_2^2 terms to be the same) generated networks with 8 hidden units consistently. This tends to indicate that CARVE is better at determining separating boundaries than Holt's method, which involves an error minimization procedure. An example of the CARVE solutions with the hyperspherical restriction is given in Fig. 8.

D. The Monks Problems

The Monks problems are a set of three artificially constructed classification tasks which were devised for use at the 2nd European Summer School on Machine Learning held in 1991 [27]. They are classification tasks in an artificial robot domain, with robots described by six attributes which are listed in Table III, along with their possible values. There are a total of 432 possible robots that can be described by these six attributes. The attributes are treated as a 17-input Boolean vector such that for each attribute value a Boolean input is given as ON if a robot has that specific attribute and OFF otherwise.

Each classification problem is specified by a logical description involving the attributes which define a class of robots that are to be identified. The training set is a portion of the set of all robots and classification performance is determined using a test set which included all possible robots. The specifications of each of the Monks problems are given in Table IV. For complete details of the specification of the training set, refer to [27].

We applied CARVE to each of the problems for ten trials. The results are given in Table V along with results of backpropagation and cascade correlation applied to the problems reported in [27]. Note that since cascade correlation produces cascade networks which have direct connections from inputs to outputs and which are completely connected from hidden units to outputs, this probably contributes to the smaller network size for cascade correlation.

Larger values of (NInit, NRot) produce near-minimal network sizes. The results of running the CARVE algorithm

TABLE II
COMPARISON OF NETWORK SIZE OF CONSTRUCTIVE ALGORITHMS ON RANDOM BOOLEAN FUNCTIONS

n	CARVE $< 100 >$	Sequential [6] $< 100 >$	Upstart [21] $< 100 >$	Oil Spot [10] $< 100 >$	Regular [8] $< 200 >$	Tiling [7] $< 25 >$
4	2.40 ± 0.69		3			3.9 (2.08 layers)
5	3.73 ± 0.58		4.5			
6	5.88 ± 0.67	7.28 ± 0.82	8	8.02 ± 1.98	15.8 ± 2.2	16.19 (3.75 layers)
7	9.47 ± 0.74		16			
8	16.23 ± 0.86	18.3 ± 0.69	33			56.98 (7.13 layers)

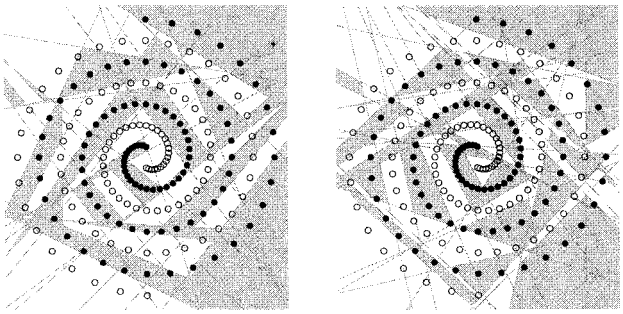

Fig. 6. Solutions to two spirals by CARVE.

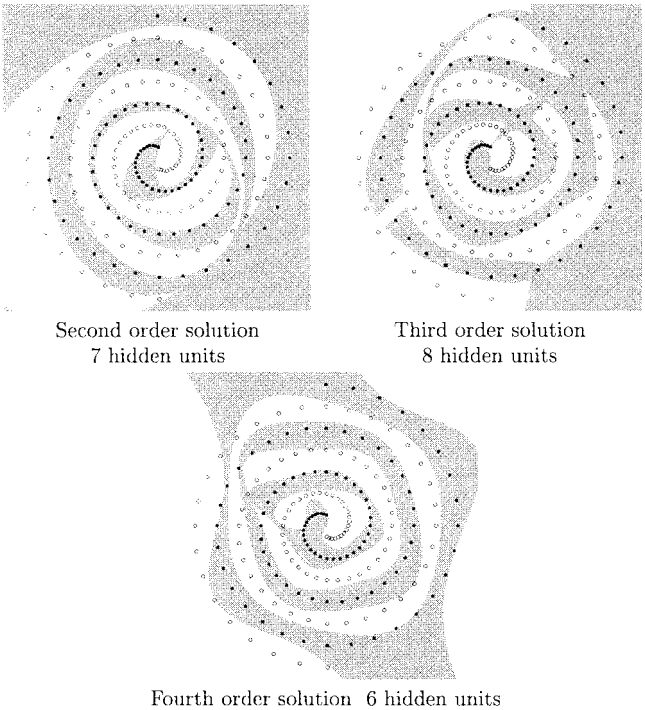

Fig. 7. Higher order solutions to two spirals.

with these larger (NInit, NRot) values are also given. The network sizes for Monks problems 1 and 2 are minimal and the generalization performance is improved for the two CARVE solutions, but backpropagation and cascade correlation still have better reported generalization performance.

For CARVE-BP, we used a learning rate of 0.001, no momentum, training for 1000 epochs. CARVE-BP produces equivalent test set performance to that reported elsewhere for BP and Cascade Correlation in [27]. Significantly for Monks problem 3, the use of CARVE provides smaller network sizes than those employed by others for BP or those constructed by cascade correlation, while still providing equivalent test set performance in the weight decay case.

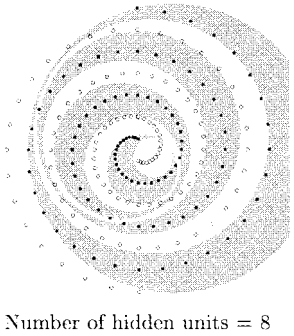

Fig. 8. Hyperspherical solution to two spirals.

TABLE III
MONKS PROBLEM ATTRIBUTE VALUES

`head shape`	round, square, octagon
`body shape`	round, square, octagon
`is smiling`	yes, no
`holding`	sword, balloon, flag
`jacket color`	red, yellow, green, blue
`has tie`	yes, no

Each of the Monks training problems were solvable using a single second-order threshold unit. For Monks problem 1, the second-order unit improves substantially upon the standard CARVE performance, but for Monks problems 2 and 3, the generalization performance of the second-order solution is worse than for that of first-order CARVE.

E. Sonar Data

Gorman and Sejnowski's sonar returns data [28] consists of 208 training examples classified into two classes (Mines/Rocks). Each training vector consists of 60 real-valued variables. Gorman and Sejnowski conducted two experiments to test generalization performance on the sonar data. The first experiment uses cross validation, splitting the 208 training examples into 13 sets of 16 examples each. Each set, in turn, is left out of the training set and is used as the test set. Ten trials on each training set are performed, and test set performance is measured. This experiment ignores the fact that the training examples come from sonar returns with different aspect angle. The second experiment takes this fact into

TABLE IV
THE MONKS PROBLEMS

- Problem M_1:(`head shape` = `body shape`) or (`jacket color` = red)

From the 432 possible examples, 124 were randomly selected for the training set, with no misclassifications.

- Problem M_2:exactly two of the six attributes have their *first* value.

From the 432 possible examples, 169 were randomly selected for the training set, with no misclassifications.

- Problem M_3:(`jacket color` is green and `holding` a sword) or (`jacket color` is not blue and `body shape` is not octagon)

From the 432 examples, 122 were randomly selected and there were 5% misclassifications.

TABLE V
MONKS PROBLEMS: HOLDOUT EXPERIMENT RESULTS

	Num. Hidden Units	Test Set perf.(%)
Monks Problem 1		
CARVE (30, 45)	3.6 ± 0.5	93.0 ± 2.6
CARVE-BP (30, 45)	3.6 ± 0.5	100.0
Backpropagation	3	100.0
Cascade Correlation	1	100.0
CARVE (135, 135)	3.0 ± 0.0	93.3 ± 3.3
CARVE-BP (135, 135)	3.0 ± 0.0	100.0
CARVE 2nd order	1.0 ± 0.0	99.1 ± 0.0
Monks Problem 2		
CARVE (35, 50)	3.6 ± 1.4	94.8 ± 5.1
CARVE-BP	3.6 ± 1.4	100.0
Backpropagation	2	100.0
Cascade Correlation	1	100.0
CARVE (175, 175)	2.0 ± 0.0	98.7 ± 1.4
CARVE-BP (175, 175)	2.0 ± 0.0	100.0
CARVE 2nd order	1.0 ± 0.0	81.9 ± 0.0
Monks Problem 3		
CARVE (45, 50)	3.6 ± 0.5	89.7 ± 1.9
CARVE-BP	3.6 ± 0.5	93.1 ± 0.8
CARVE-BP with wgt decay	3.6 ± 0.5	97.2 ± 0.0
Backpropagation	4	93.1
BP with wgt decay	4	97.2
Cascade Correlation	3	95.4
CC with output decay	3	97.2
CARVE (120, 120)	2.7 ± 0.5	91.5 ± 1.2
CARVE-BP (120, 120)	2.7 ± 0.5	92.9 ± 1.0
CARVE-BP with wgt decay	2.7 ± 0.5	97.2 ± 0.0
CARVE 2nd order	1.0 ± 0.0	82.9 ± 0.0

TABLE VI
SONAR DATA EXPERIMENT 1: ASPECT ANGLE INDEPENDENT SERIES

	Network Size	Training set perf.(%)	Test Set perf.(%)
CARVE (25, 65)	6.9 ± 0.5	100	81.2 ± 10.1
CARVE-BP (25, 65)	6.9 ± 0.5	95.3 ± 0.3	84.2 ± 8.2
CARVE 2nd Order (25, 25)	8.1 ± 0.6	100	76.9 ± 9.7
BP	0	89.4 ± 2.1	77.1 ± 8.3
	2	96.5 ± 0.7	81.9 ± 6.2
	3	98.8 ± 0.4	82.0 ± 7.3
	6	99.7 ± 0.2	83.5 ± 5.6
	12	99.8 ± 0.1	84.7 ± 5.7
	24	99.8 ± 0.1	84.5 ± 5.7

account and splits the data set into equal sized training and test sets with examples from each aspect angle. The generalization performance over ten trials is averaged. We carried out Gorman and Sejnowski's experiments using CARVE instead of backpropagation and the results are presented in Tables VI and VII.

The network size at which 100% training set performance is achieved by CARVE is significantly smaller than the size

TABLE VII
SONAR DATA EXPERIMENT 2: ASPECT ANGLE DEPENDENT SERIES

	Network Size	Training set perf.(%)	Test Set perf.(%)
CARVE (100, 100)	3.2 ± 0.4	100	79.5 ± 3.8
CARVE-BP (100, 100)	3.2 ± 0.4	98.1 ± 2.1	85.1 ± 0.02
CARVE 2nd Order (50, 45)	4.5 ± 0.5	100	78.7 ± 4.5
BP	0	79.3 ± 3.4	73.1 ± 4.8
	2	96.2 ± 2.2	85.7 ± 6.3
	3	98.1 ± 1.5	87.6 ± 3.0
	6	99.4 ± 0.9	89.3 ± 2.4
	12	99.8 ± 0.6	90.4 ± 1.8
	24	100	89.2 ± 1.4

required by the backpropagation networks. In fact in experiment 1, backpropagation fails entirely to achieve 100% training set performance. For experiment 2, since a single run was only for ten trials (rather than 130 trials in experiment 1) we ran experiments for higher values of (NInit, NRot) to find the minimal network size that CARVE can achieve on the task. As the amount of search increased the network sizes achieved by CARVE decreased and the generalization performance increased, in a similar fashion to CARVE's behavior on the Monk's problems. Nevertheless, the test set performance for CARVE in both experiments is consistently worse than that achieved by backpropagation (except for backpropagation networks with no hidden units). Note also that second-order CARVE exhibited performance that was marginally inferior to the first-order version, both in terms of network size and generalization performance. A difficulty faced by the second-order algorithm is the size of the second-order vectors generated from the sonar data. The 60-dimension input vector for the sonar data becomes a second-order vector with 1891 terms, so that the second-order algorithm is unable to find smaller network solutions in the same time as the first-order algorithm.

CARVE-BP provided an improvement in test set performance compared with CARVE alone. We used a learning rate of 0.0001, momentum of 0.7 for both experiments. For experiment 1, we trained over 10 000 epochs, for experiment 2, 100 000 epochs. In experiment 1, the test set performance of CARVE-BP approaches those of the best BP runs with smaller networks generally being used by the CARVE-BP solutions, but for experiment 2, in the application of CARVE-BP, the test set performance is comparable to the lower BP results, but doesn't achieve the same level as the best BP runs. Notice that for the CARVE-BP solutions for the sonar data, we didn't achieved 100% training set performance. For very large weight factors (greater than 5000), the training set performance was

TABLE VIII
IRIS DATA: CROSS VALIDATION LEAVE ONE OUT EXPERIMENT RESULTS

	Network Size	Training set error	Cross Validation error
CARVE	3.31 ± 0.96	0.0	0.027
Offset	3.41 ± 1.01	0.0	0.033
BP	3	0.017	0.033
CARVE 2nd order	2.0 ± 0.0	0.0	0.053

100%, but on the test set no improvement to the CARVE performance was obtained.

F. Iris Data

The iris database is a well used pattern classification benchmark [29], [30]. The data consists of 150 patterns of four real-valued attributes, which measure the width and length of the sepals and petals in 50 examples of three different iris flower types. In common with the methods with which we compare, we measured cross validation error with the leave-one-out scheme. The results are tabulated in Table VIII. CARVE improves upon the performance of Martinez and Esteve's [31] "offset algorithm" in terms of network size (even though the offset algorithm constructs threshold networks with two hidden layers), and has improved generalization performance over both the offset algorithm and backpropagation [30]. The second-order CARVE algorithm is able to find the minimal network to separate the three iris classes (indicating that the iris data is second-order separable), but the generalization performance is worse than all the first-order methods.

V. DISCUSSION AND CONCLUSIONS

The CARVE algorithm builds threshold networks with a single hidden layer which correctly implement consistent classification tasks on real-valued training sets. The only condition required of the training tasks for CARVE to operate correctly is that the tasks be consistent, i.e., that any single training example belongs to one class only. The algorithm is an explicit construction method which builds the hidden layer one unit at a time. Each hidden unit separates a carve set from the training set. The construction is guaranteed to finish and will produce a hidden layer with faithful and linearly separable internal representations.

The task of finding largest carve sets is NP-hard, but we have presented an approach that allows good-sized carve sets to be found efficiently. This was achieved by modifying the gift-wrapping method to produce a method of searching a convex hull of arbitrary dimension and hence to find carve sets. In comparison with the sequential learning algorithm [6], noting the improved performance in network size for random Boolean functions (Section IV-B), CARVE exhibits improved performance in the size of carve sets that it finds. The time complexity for this method is proportional to NInit $\times$ NRot (which are user defineable parameters) and is $O(n^2 \log n + n^2 d)$. Since the time complexity of our method is not exponential in the dimension d, it is feasible to extend the method to higher order threshold networks, where the dimensionality of vectors can become large.

Our experimental results demonstrate that the algorithm produces known minimal sized solutions for n-bit parity and produces smaller networks than those reported for other constructive algorithms on the random Boolean functions problem.

For the classification tasks where we considered generalization performance, CARVE's performance compares less favorably with other reported results. A likely reason for the weaker generalization performance of the CARVE algorithm is that the algorithm always produces networks that correctly classify all the training data, and so there is a greater danger of overfitting to the training data.

CARVE-BP produced improved generalization performance which was comparable to the application of BP in the examples we considered. Using CARVE to establish initial networks for the training tasks we considered provided smaller networks than those previously reported for these training tasks. This suggests that CARVE provides a useful method for determining appropriate network sizes for classification problems.

Second-order CARVE was seen to produce network solutions which better fit the two spirals problem, but for the Monks, Sonar, and Iris problems it did not yield any marked improvement in generalization performance. One reason for this failure to improve generalization performance could be that the expressive power of second-order units is greater than that of first-order units, and this causes the second-order network solutions to suffer more from the problem of overfitting.

For the Monks problems, we were able to employ the network size behavior of the CARVE algorithm to improve generalization performance. By estimating (NInit, NRot) values which produced minimal networks for the training task, the generalization performance was improved. This is in agreement with the widely-accepted view that smaller networks tend to have better generalization performance than larger ones.

REFERENCES

[1] J. Sietsma and R. J. F. Dow, "Creating artificial neural networks that generalize," *Neural Networks*, vol. 4, pp. 67–79, 1991.
[2] Y. L. Cun, J. S. Denker, and S. A. Solla, "Optimal brain damage," *Advances in Neural Information Processing Systems (NIPS)*, vol. 2, pp. 598–605, 1990.
[3] B. Hassibi and D. G. Stork, "Second-order derivatives for network pruning: Optimal brain surgeon," *Advances in Neural Information Processing Systems (NIPS)*, vol. 5, pp. 164–171, 1993.
[4] F. J. Śmieja, "Neural network constructive algorithms: Trading generalization for learning efficiency?," *Circuits Syst. Signal Processing*, vol. 12, no. 2, pp. 331–374, 1993.
[5] E. Fiesler, "Comparative bibliography of ontogenic neural networks," in *Proc. Int. Conf. Artificial Neural Networks (ICANN'94)*, 1994, pp. 793–796.
[6] M. Marchand, M. Golea, and P. Ruján, "A convergence theorem for sequential learning in two-layer perceptrons," *Europhys. Lett.*, vol. 11, no. 6, pp. 487–492, 1990.

[7] M. Mézard and J-P. Nadal, "Learning in feedforward layered networks: The tiling algorithm," *J. Phys. A: Math. General*, vol. 22, pp. 2191–2203, 1989.
[8] P. Ruján and M. Marchand, "Learning by minimizing resources in neural networks," *Complex Syst.*, vol. 3, pp. 229–241, 1989.
[9] G. T. Barkema, H. M. A. Andree, and A. Taal, "The patch algorithm: Fast design of binary feedforward neural networks," *Network*, vol. 5, pp. 393–407, 1993.
[10] F. M. Frattale-Mascioli and G. Martinelli, "A constructive algorithm for binary neural networks: The oil-spot algorithm," *IEEE Trans. Neural Networks*, vol. 6, pp. 794–797, 1995.
[11] M. Muselli, "On sequential construction of binary neural networks," *IEEE Trans. Neural Networks*, vol. 6, pp. 678–690, 1995.
[12] B. Widrow and M. A. Lehr, "30 years of adaptive neural networks: Perceptron madaline and backpropagation," *Proc. IEEE*, vol. 78, no. 9, pp. 1415–1442, Sept. 1990.
[13] L. Breiman, J. H. Friedman, R. A. Oshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.
[14] M. Marchand and M. Golea, "On learning simple neural concepts: From halfspace intersections to neural decision lists," *Newtork*, vol. 4, pp. 67–85, 1993.
[15] D. S. Johnson and F. P. Preparata, "The densest hemisphere problem," *Theoretical Comput. Sci.*, vol. 6, pp. 93–107, 1978.
[16] R. A. Jarvis, "On the identification of the convex hull of a finite set of points in the plane," *Inform. Processing Lett.*, vol. 2, pp. 18–21, 1973.
[17] D. R. Chand and S. S. Kapur, "An algorithm for convex polytopes," *J. Assoc. Comput. Machinery*, vol. 17, no. 1, pp. 78–86, Jan. 1970.
[18] G. Swart, "Finding the convex hull facet by facet," *J. Algorithms*, vol. 6, pp. 17–48, 1985.
[19] S. Young, "Constructive neural-network training algorithms for pattern classification using computational geometry techniques," Ph.D. dissertation, Dept. Electr. Comput. Eng., Univ. Queensland, 1995.
[20] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning internal representations by error propagation," *Parallel Distributed Processing*, D. E. Rumelhart and G. McClelland, Eds. Cambridge, MA: MIT Press, 1986, ch. 8, pp. 318–632.
[21] M. Frean, "The upstart algorithm: A method for constructing and training feedforward neural networks," *Neural Comput.*, vol. 2, pp. 198–209, 1990.
[22] K. J. Lang and M. J. Witbrock, "Learning to tell two spirals apart," in *Proc. 1988 Connectionist Models Summer School*, 1989, pp. 52–61.
[23] S. E. Fahlman and C. Lebiere, "The cascade-correlation learning architecture," *Advances Neural Inform. Processing Syst. (NIPS)*, San Mateo, CA, vol. 2, pp. 524–532, 1990.
[24] E. B. Baum and K. J. Lang, "Constructing hidden units using examples and queries," *Advances Neural Inform. Processing Syst. (NIPS)*, vol. 2, pp. 904–910, 1990.
[25] M. J. J. Holt, "Sequential learning of casasent network classifiers: Constructing piecewise spherical decision boundaries," in *Proc. Int. Conf. Artificial Neural Networks (ICANN'94)*, 1994, pp. 1412–1415.
[26] ———, "A weight learning strategy for sequential construction of casasent network classifiers," in *Pro. Int. Conf. Artificial Neural Networks (ICANN'94)*, 1994, pp. 1416–1419.
[27] S. B. Thrun, J. Bala, E. Bloedorn, I. Bratko, B. Cestnik, J. Cheng, K. D. Jong, S. Džeroksi, S. E. Fahlman, D. Fisher, R. Hamann, K. Kaufman, S. Keller, I. Kononenko, J. Kreuziger, R. S. Michalski, T. Mitchell, P. Pachowicz, Y. Reich, H. Vafaie, W. V. de Welde, W. Wenzel, J. Wnek, and J. Zhang, "The MONK's problems: A performance comparison of different learning algorithms," Carnegie Mellon Univ., CME-CS-91-197, Dec. 1991.
[28] R. P. Gorman and T. J. Sejnowski, "Analysis of hidden units in a layered network trained to classify sonar targts," *Neural Networks*, vol. 1, pp. 75–89, 1988.
[29] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Ann. Eugenics*, vol. VII, no. pt. II, pp. 179–188, 1936.
[30] S. M. Weiss and I. Kapouleas, "An empirical comparison of pattern recognition, neural nets, and machine learning classification methods," in *Proc. 11th Int. Joint Conf. Artificial Intell. (IJCAI'89)*, 1989, pp. 781–787.
[31] D. Martinez and D. Estève, "The offset algorithm: Building and learning method for multilayer neural networks," *Europhys. Lett.*, vol. 18, no. 2, pp. 95–100, 1992.

Steven Young received the B.E. (Hons), B.Sc., and Ph.D. degrees from the University of Queensland, Australia, in 1990, 1990, and 1996, respectively.

From 1995 to 1996, he was a Research Scientist in the Department of Experimental Psychology at the University of Oxford, U.K. He is currently the Computer Officer for the Centre for Cognitive Neuroscience at the University of Oxford. His research interests include neural networks, computational complexity, computational geometry, and brain imaging.

Thomas Downs (M'74) received the B.Tech. and Ph.D. degrees from the University of Bradford, U.K., in 1968 and 1972, respectively.

From 1968 to 1973 he was employed by the Marconi Company in Chelmsford, U.K., in the Theoretical Sciences Laboratory, where he worked on the computer-aided design of electrical circuits and systems. In 1973, he joined the Department of Electrical Engineering at the University of Queensland, Australia, where he is now Professor. His main research interests include theory and application of artificial neural networks, mathematical statistics, and applied probability modeling. He has published approximately 120 technical papers and is coauthor of the book *Logic Design with Pascal* (New York: Van Nostrand Reinhold, 1988).